

CAGOULE

Cryptographie Algébrique Géométrique par Ondes et Logique Entrelacée
VERSION 2.2.0 — ROADMAP TECHNIQUE

Document	CAGOULE v2.2.0 — Roadmap Technique
Auteur	Slim Issa
Version cible	v2.2.0
Baseline	v2.1.0 (1er mai 2026)
Dépôt	github.com/slimissa/CAGOULE
Licence	MIT
Plateforme cible	x86_64 Python 3.12+ Linux (AVX2 requis pour SIMD)

✔ **Statut v2.1.0** : Tous les objectifs de la version 2.1.0 sont atteints. Ce document planifie la version suivante.

1. Contexte et Objectifs de v2.2.0

1.1 Bilan de v2.1.0

CAGOULE v2.1.0 a atteint ses trois objectifs en un cycle de version :

- **Portage C de omega.c** : génération des 64 clés de ronde $\zeta(2n) \rightarrow$ round keys entièrement en C, éliminant mpmath des chemins de production.
- **Correction sécurité test_mauvais_mdp** : decrypt() re-dérive systématiquement les paramètres depuis (password, salt_cgl1), rendant toute tentative avec un mauvais mot de passe détectable via CagouleAuthError.
- **Enrichissement des exceptions** : CagouleAuthError et CagouleFormatError disposent désormais d'attributs de diagnostic (.reason, .hint, .ct_size, .field, .min_size).

Les résultats de cagoule-bench v1.0.0 confirment un débit de 23,4 MB/s (+568 % vs v2.0), une latence de 42,8 ms (−85 %), et une empreinte mémoire de 3,2 MB (−96 %). La symétrie chiffrement/déchiffrement est parfaite (∼1×).

1.2 Objectifs de v2.2.0

La version 2.2.0 vise trois axes complémentaires :

Axe	Priorité	Impact attendu
SIMD / AVX2 — vectorisation mulmod64 et Vandermonde	☐ Critique	>30 MB/s (+25–30 %)
Ergonomie C/Python — DiffusionMatrixC.free() déterministe	☐ Important	Stabilité long-durée
Benchmark multiprocessing (ProcessPoolExecutor)	☐ Important	Validation parallélisme
Intégration QuantOS — primitives QShell encrypt/decrypt	☐ Stratégique	Convergence QuantOS
PyPI packaging — manylinux wheel .so pré-compilé	☐ Important	Distribution zero-build
Audit externe — revue par les pairs / NIST	☐ Long terme	Validation académique
Correction pytest — 2 failures de linkage libcagoule.so	☐ Critique	560/560 tests verts

2. SIMD et Vectorisation AVX2

Objectif principal : Porter le pipeline CBC de CAGOULE de ~23 MB/s à >30 MB/s via des instructions AVX2 (256-bit), sans modifier le format CGL1.

2.1 Analyse du goulot d'étranglement

Le pipeline CBC-like de CAGOULE traite actuellement les 16 éléments d'un bloc de manière séquentielle. Chaque élément est un `uint64_t` dans `Z/pZ`. Les deux opérations dominantes en cycles CPU sont :

- **mulmod64(a, b, p) :** multiplication modulaire 64 bits utilisant `__uint128_t`. C'est le calcul le plus coûteux — il se produit $16 \times 16 = 256$ fois par bloc pour la multiplication Vandermonde.
- **Multiplication matricielle Vandermonde :** la matrice 16×16 multiplie chaque bloc, soit 256 `mulmod64` + 256 `addmod64` par bloc. C'est le hotspot mesuré lors du profiling v2.0/v2.1.

L'introduction d'AVX2 permet de traiter 4 éléments `uint64_t` en parallèle dans un registre `__m256i` (256-bit), réduisant le nombre d'itérations séquentielles par un facteur 4.

2.2 Plan d'implémentation

2.2.1 Vectorisation de mulmod64 (__m256i)

Le nouveau `cagoule_math_avx2.h` exposera `mulmod64x4()`, traitant 4 paires (a_i, b_i) simultanément via des intrinsèques AVX2 :

Intrinsèques clés : `_mm256_mul_epu32`, `_mm256_srli_epi64`, `_mm256_add_epi64`, `_mm256_sub_epi64`, `_mm256_cmpgt_epi64` (réduction Barrett ou Montgomery 64-bit).

- **Réduction modulaire vectorielle :** la méthode de Barrett modifiée pour `__m256i` évite le débordement et s'avère plus rapide que `__uint128_t` scalaire pour des lots de 4.
- **Fichier cible :** `cagoule_math_avx2.h` — inclus conditionnellement selon la disponibilité AVX2 détectée à la compilation ET au runtime.
- **Tests C associés :** ajout de `test_math_avx2.c` couvrant la correction bit-à-bit contre la référence scalaire (≥ 128 vecteurs de test), la bijectivité, et le bench comparatif.

2.2.2 Vectorisation de la multiplication Vandermonde

La boucle interne de `cagoule_matrix_mul()` sera restructurée pour traiter 4 lignes de la matrice simultanément :

- Chargement de 4 lignes $\times$ 16 colonnes via `_mm256_loadu_si256`
- Produit vectoriel avec `mulmod64x4()` sur les 16 éléments de colonne
- Réduction horizontale (`hadd` équivalent `mod p`) pour assembler les 4 résultats
- Stockage du résultat avec `_mm256_storeu_si256`

La matrice inverse (pour le déchiffrement) utilise la même structure — la symétrie Feistel garantit que la vectorisation s'applique identiquement à l'encryption et à la decryption.

2.2.3 Détection runtime AVX2 — fallback scalaire

La détection AVX2 sera effectuée à deux niveaux pour garantir la portabilité :

Niveau	Mécanisme	Comportement
Compilation	-mavx2 / __AVX2__ macro	Inclusion conditionnelle de cagoule_math_avx2.h
Runtime	__builtin_cpu_supports("avx2")	Sélection de la fonction vectorielle ou scalaire
Fallback	Pointeur de fonction statique	cagoule_matrix_mul_scalar() si AVX2 absent
Python	ctypes.cdll + attribut backend_info	Exposer le backend actif ("avx2" / "scalar")

Le fallback scalaire est identique au code de v2.1.0 — aucune régression sur les machines sans AVX2 (VMs, anciens processeurs, ARM via émulation).

2.2.4 Compatibilité binaire CGL1

Invariant de format : La vectorisation est purement interne au moteur C. Le format CGL1 (MAGIC|VERSION|SALT|NONCE|CIPHERTEXT|TAG) reste inchangé. Tout ciphertext v2.1.0 est déchiffrable par v2.2.0 et vice versa.

2.3 Gains attendus

Scénario	v2.1.0 (scalaire)	v2.2.0 (AVX2)	Gain estimé
mulmod64 seul (×4 lanes)	~8 cycles/op	~2–3 cycles/op	×2,5–4
Matrix mul Vandermonde 16×16	256 mulmod64 séquentiels	64 mulmod64x4	×3–4
Throughput encrypt 1 MB	~23,4 MB/s	>30 MB/s (objectif)	+25–30 %
Latence encrypt 1 MB	~42,8 ms	<34 ms (objectif)	–20–25 %

Note : Les gains dépendent du CPU (microarchitecture, largeur de bande mémoire). Les benchmarks seront exécutés sur Intel Skylake/Zen2+ qui supportent AVX2 nativement. cagoule-bench v2.0.0 inclura un mode --avx2-only pour mesurer le delta sans bruit système.

2.4 Tests et Validation

- test_math_avx2.c :** correction bit-à-bit mulmod64x4 vs mulmod64 scalaire — 512 cas générés aléatoirement.

- **test_matrix_avx2.c** : roundtrip $P \times P^{-1} = I \bmod p$ en mode AVX2, même vecteurs KAT que test_matrix.c.
- **Parity test** : `encrypt(avx2) == encrypt(scalar)` pour 100 messages aléatoires de 1 KB à 1 MB — validation de l'identité des résultats.
- **CI Makefile** : cible `make test-avx2` ajoutée ; `make tests` continue de pointer vers les tests scalaires (CI sans AVX2).

3. Ergonomie C/Python — DiffusionMatrixC.free()

3.1 Problème identifié

En v2.1.0, la libération mémoire de DiffusionMatrixC repose sur `__del__`. L'ordre de destruction des objets Python en fin de programme n'est pas garanti par CPython. Si `_lib` est collecté avant la matrice, l'appel ctypes à la fonction de libération C peut lever une `AttributeError` silencieuse ou un `segfault` dans des programmes long-durée.

Impact : Applications long-durée (serveurs, daemons) ou scripts créant/détruisant de nombreuses matrices. Risque faible en usage normal, mais inacceptable pour une intégration QuantOS.

3.2 Solution

- **Exposer DiffusionMatrixC.free()** : méthode Python déterministe appelant explicitement la fonction C de libération, indépendamment de `__del__`.
- **Context manager** : implémenter `__enter__` / `__exit__` pour permettre l'usage `with DiffusionMatrixC(...) as mat:` — garantit le `free()` même en cas d'exception.
- **Guard double-free** : flag `_freed = False`, levant `RuntimeError` si `free()` est appelé deux fois.
- **Tests** : ajout de 8 tests dans `test_matrix.py` couvrant `free()` explicite, context manager, double-free guard, et comportement post-free.

4. Benchmark Mode ProcessPoolExecutor

4.1 Objectif

Mesurer le throughput de CAGOULE en parallélisme multiprocess (Python ProcessPoolExecutor) pour valider la thread-safety de `libcagoule.so` et quantifier le gain de scalabilité horizontale. Ce benchmark est distinct du benchmark single-core de `cagoule-bench v1.0.0`.

4.2 Plan

- **cagoule-bench v2.0.0 — mode --parallel** : nouveau flag activant ProcessPoolExecutor avec N workers (1, 2, 4, 8).
- **Métriques mesurées** : throughput agrégé (MB/s total), throughput par worker, efficacité parallèle ($\text{ratio throughput}_N / N \times \text{throughput}_1$), latence p95 par worker.
- **Thread-safety libcagoule.so** : les tests de concurrence vérifieront l'absence de data race via Helgrind (Valgrind) — aucune variable globale mutable n'est prévue dans le backend C.
- **Rapport** : ajout d'une section "Parallel Benchmark" dans `cagoule-bench v2.0.0` avec tableau et graphique ASCII.

5. Intégration QuantOS

Contexte : QuantOS est une plateforme de computing orientée finance quantitative. QShell est son shell natif doté d'opérateurs de pipeline (`|>`, `?>`) et d'aliases de trading. CAGOULE v2.2.0 doit s'intégrer comme primitive cryptographique de première classe.

5.1 Primitives QShell

Deux commandes QShell seront exposées via `cagoule-pass` :

Commande QShell	Action	Exemple d'usage
<code>cgl encrypt <file></code>	Chiffre un fichier, sortie CGL1	<code>universe_data.csv > cgl encrypt > universe_data.cgl</code>
<code>cgl decrypt <file></code>	Déchiffre un fichier CGL1	<code>cgl decrypt < universe_data.cgl > momentum_filter</code>
<code>cgl bench</code>	Lance <code>cagoule-bench</code> et affiche le throughput	<code>cgl bench --size 1MB</code>
<code>cgl info <file></code>	Affiche les métadonnées CGL1 (salt, nonce, version)	<code>cgl info signal_output.cgl</code>

5.2 Intégration opérateur `|>` (pipeline JSON)

Le pipeline JSON de QShell (`universe` → `momentum_filter` → `risk_filter` → `size_positions` → `execute`) pourra chiffrer/déchiffrer les étapes sensibles en ligne :

Exemple de pipeline sécurisé : `universe | cgl encrypt | risk_filter | cgl decrypt | size_positions | execute`

Cette intégration nécessite que `cgl encrypt/decrypt` supportent les modes `stdin/stdout` pour être compatibles avec l'opérateur pipe de QShell.

5.3 `cagoule-pass` — gestionnaire de mots de passe

`cagoule-pass` v2.0.0 intégrera la nouvelle API v2.2.0 :

- **Bénéfice AVX2 :** déchiffrement du vault ~25 % plus rapide (perceptible pour les vaults > 500 entrées).
- **DiffusionMatrixC.free()** : `cagoule-pass` est un daemon long-durée — le fix `__del__` est directement bénéfique.
- **Commande `qshell vault`** : nouvelle commande QShell pour accéder à `cagoule-pass` sans quitter le shell.

6. PyPI Packaging — manylinux Wheel

6.1 Objectif

Permettre l'installation de CAGOULE via `pip install cagoule` sans nécessiter de compilation C locale. La distribution inclura des binaires .so pré-compilés pour les principales plateformes Linux.

6.2 Plan technique

Étape	Description	Outil
pyproject.toml	Finaliser la configuration build : metadata, entry-points, dépendances	setuptools / flit
Extension C	Déclarer libcagoule.so comme Extension dans setup.py	cffi / ctypes wheel
manylinux build	Build dans manylinux2014_x86_64 (GCC 8+, OpenSSL 1.1)	cibuildwheel / docker
Wheel AVX2	Wheel séparée ou détection runtime embarquée	cibuildwheel + tags
Upload TestPyPI	Validation avant publication PyPI officielle	twine
CI GitHub Actions	Build + test automatique à chaque tag v*.*.0	GitHub Actions

6.3 Dépendances pré-packagées

Les binaires .so pré-compilés embarqueront libcrypto (OpenSSL) en version statique pour éviter les conflits de version sur l'hôte. argon2-cffi restera une dépendance Python normale (déjà disponible sur PyPI).

7. Audit Externe — Revue par les Pairs

Contexte : La garantie de sécurité de CAGOULE repose sur ChaCha20-Poly1305 (RFC 8439). La couche algébrique (Vandermonde + Feistel + ζ -round keys) n'a pas encore été soumise à une revue formelle. Cette section planifie les étapes vers une validation académique.

7.1 Périmètre de l'audit

- **Construction Vandermonde sur Z/pZ** : propriétés de diffusion (MDS bound), résistance aux attaques différentielles et linéaires.
- **S-box Feistel (cycle-walking)** : bijectivité, non-linéarité, résistance aux attaques algébriques.
- **ζ -round keys via HKDF** : indépendance des clés de ronde, entropie, résistance aux attaques de distingueur.
- **Pipeline complet CBC-like + AEAD** : sécurité de la composition avec ChaCha20-Poly1305.

7.2 Livrables préparatoires

Avant de soumettre à une revue externe, les documents suivants seront produits :

- Spécification formelle de la couche algébrique (notation mathématique rigoureuse)
- Preuves de bijectivité S-box et de propriété MDS de la matrice Vandermonde
- Vecteurs KAT complets avec justification de la construction
- Résultats des 14 tests statistiques NIST SP 800-22 (section `test_nist.py` — 0 skip)

7.3 Cibles de soumission

Cible	Type	Pertinence
IACR ePrint Archive	Preprint public	Visibilité rapide, retours informels
CryptoSym (CHES/FSE)	Conférence académique	Revue formelle par les pairs
Crypto Stack Exchange	Communauté	Retours rapides sur la construction
NIST Lightweight Cryptography	Standardisation (hors scope direct)	Si CAGOULE vise un usage embarqué

8. Correction des 2 Failures Pytest

8.1 Diagnostic

Les 2 failures et 2 errors de la session pytest v2.1.0 (523/560 passés) sont liés à l'environnement de compilation de `libcagoule.so` — non à la logique cryptographique. Le diagnostic v2.2.0 visera à identifier la cause racine :

Hypothèse	Symptôme	Action corrective
libcagoule.so absent du PATH Python	ImportError ou OSError au chargement ctypes	Corriger make install + RPATH dans setup.py
Version libcagoule.so incompatible	Symbol not found ou wrong ELF class	Vérifier les tags de version dans l'ABI C
OpenSSL absent dans l'env de test CI	Linker error sur HMAC_CTX_new	Bundler OpenSSL statique dans le wheel de test
Test d'isolation de contexte	État partagé entre tests	Ajouter fixtures pytest d'isolation

8.2 Objectif

Cible v2.2.0 : 560/560 tests Python passés — 0 failures, 0 errors, 0 skips liés à l'environnement. Les skips NIST (timeout) restent acceptables uniquement s'ils sont explicitement documentés.

9. Planning et Livrables

9.1 Séquençage des travaux

Phase	Travaux	Durée estimée	Prérequis
P0 — Hotfix	Correction 2 failures pytest (linkage libcagoule.so)	1–2 jours	—
P1 — AVX2 Math	cagoule_math_avx2.h + test_math_avx2.c + validation bit-à-bit	3–5 jours	P0
P2 — AVX2 Matrix	Vectorisation cagoule_matrix_mul() + tests + bench	3–4 jours	P1
P3 — AVX2 Runtime	Détection runtime + fallback scalaire + CI Makefile	1–2 jours	P2
P4 — Ergonomie	DiffusionMatrixC.free() + context manager + 8 tests	1–2 jours	P0
P5 — Bench Parallel	cagoule-bench v2.0.0 --parallel + Helgrind	2–3 jours	P3
P6 — QuantOS	cgl encrypt/decrypt QShell + stdin/stdout + intégration cagoule-pass	3–4 jours	P3, P4
P7 — PyPI	pyproject.toml + manylinux wheel + CI GitHub Actions	3–5 jours	P3, P4
P8 — Audit	Spécification formelle + preuves + soumission IACR ePrint	Continu	P7

9.2 Livrables finaux de v2.2.0

Livable	Description
cagoule_math_avx2.h	Intrinsèques AVX2 pour mulmod64x4 avec fallback scalaire
cagoule_matrix_avx2.c	Multiplication Vandermonde vectorisée (4 lignes simultanées)
cagoule-bench v2.0.0	Benchmark multi-taille + mode --parallel + section AVX2
DiffusionMatrixC.free()	Méthode Python déterministe + context manager + guard double-free
Primitives QShell cgl	cgl encrypt / decrypt / bench / info via cagoule-pass v2.0.0
PyPI wheel manylinux	pip install cagoule fonctionnel sans compilation C locale
Suite pytest 560/560	Zéro failure/error liés à l'environnement
Suite C 256+ tests	Nouveaux tests AVX2 (≥50) + tests DiffusionMatrixC.free()
Rapport Technique v2.2.0	Ce document mis à jour avec les résultats réels
Spécification formelle algèbre	Document préparatoire pour soumission IACR ePrint

10. Analyse de Sécurité — Impact v2.2.0

10.1 Invariants de sécurité conservés

La vectorisation AVX2 est une optimisation pure du moteur d'exécution. Les invariants cryptographiques suivants sont inchangés :

- Format CGL1 identique — aucun changement de MAGIC, VERSION, SALT, NONCE, TAG.
- Primitives cryptographiques inchangées — ChaCha20-Poly1305 (RFC 8439), Argon2id (RFC 9106), HKDF-SHA256 (RFC 5869).
- Résultats bit-à-bit identiques — `encrypt_avx2(msg, pwd) == encrypt_scalar(msg, pwd)` pour tout (msg, pwd).
- Zéroisation des clés — volatile uint8_t* conservée dans tous les chemins AVX2.

10.2 Nouvelles surfaces d'attaque potentielles

Surface	Risque	Mitigation
Registres YMM (AVX2)	Residual key material dans les registres après opération	VZEROALL / VZEROUPPER en fin de fonction critique
Timing side-channel AVX2	Variabilité de latence selon l'état de puissance du CPU	Opérations sur données uniformes — pas de branchement data-dépendant
Fallback scalaire non-testé en CI AVX2	Régression silencieuse du fallback	CI dédié make test-scalar force le mode scalaire même sur CPU AVX2

11. Conclusions

CAGOULE v2.2.0 représente l'aboutissement de la migration vers un backend C haute performance. Après avoir porté la couche algébrique (v2.0) et le générateur de clés de ronde omega.c (v2.1), la vectorisation AVX2 constitue le dernier levier d'optimisation majeur avant d'atteindre les limites de la bande passante mémoire sur x86_64.

Les autres travaux (DiffusionMatrixC.free(), packaging PyPI, intégration QuantOS) transforment CAGOULE d'un projet de recherche en bibliothèque prête pour la production. La soumission IACR ePrint marquerait une étape de maturité supplémentaire.

Objectif cible v2.2.0 : >30 MB/s throughput | 560/560 tests verts | pip install cagoule | cgl encrypt/decrypt dans QShell | Spécification formelle publiée.

Slim Issa — Kairouan, Tunisie — Mai 2026
github.com/slimissa/CAGOULE